Adaptive Decoding via Latent Preference Optimization

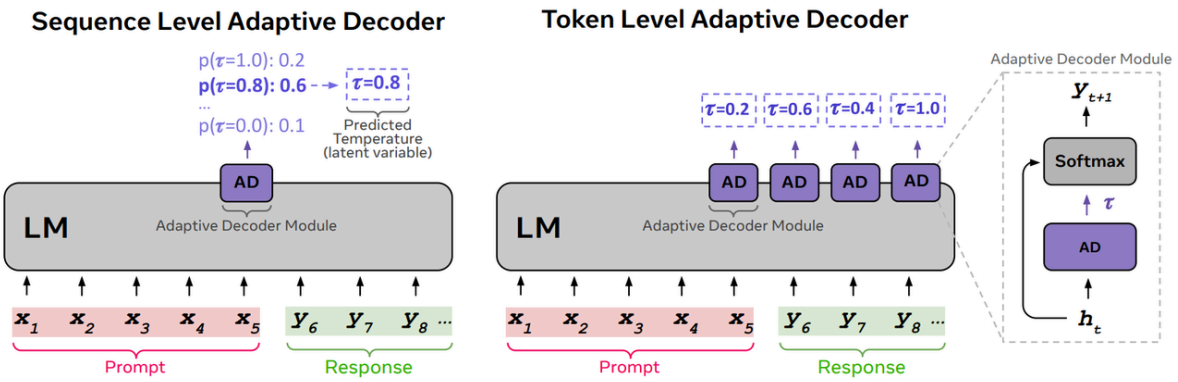
> #Architecture #Finetuning

- What problems does this project solve?
  - In the traditional LLM, the temperature is a hyperparameter. No matter what tasks it handles, the temperature will not change during the inference time.
  - However, it is observed that not all tasks can use the same temperature (for example, 0.7). Instead, the temperature decision should be dynamic, even token-level dynamic, based on different tasks to be handled.
- What does this architecture do?
  - Intuition: Add a module to the model to predict a distribution of temperature and choose the temperature (by sampling|greedily) from the distribution.
  - Before softmax-ing the logits, we add a differentiable module (for example, MLP) to estimate a distribution of temperature based on the current hidden state:

    $P(\tau_k|h_t) = \text{ADAPTIVEDECODER}(h_t)$
  - Then, from the distribution, select a temperature by either:
    1. Greedily:

       $\tau = \text{argmax}_{\tau_k} P(\tau_k|h_t)$
    2. Sampling:

    $\tau \sim P(\tau_k|h_t)$
  - Then use the selected temperature to generate.
  - 2 type of decision:



    1. Token level: Before generating each token, call the module once to get a token-specific temperature.
    2. Sequence level: The temperature is determined during the prompt-process stage; then it's fixed.
  - **Finetuning**: The finetuning of the temperature module is based on DPO. There are 3 strategies:
    - Regard the temperature as a token:

      $$\mathcal{L}_{LPO} = -\log\sigma\left[\beta\log\frac{P(y^c)}{P_{ref}(y^c)} - \beta\log\frac{P(y^r)}{P_{ref}(y^r)} + \beta\log P(\tau^c) - \beta\log P(\tau^r)\right]$$

        - $c = \text{token}, r = \text{reject}$
        - Reference model has no temperature module, regard it as a constant.
    - Ignore the token, only consider the difference between preferred and rejected temperature (*used in the project*).

      $$\mathcal{L}_{LPO} = -\log\sigma\left[\beta\log P(\tau^c) - \beta\log P(\tau^r)\right]$$
    - Regard the temperature as a latent variable:

      $$\mathcal{L}_{LPO} = -\log\sigma\left[\beta\sum_t\log\frac{\sum_\tau P(y_t^c|\tau)P(\tau)}{\sum_\tau P_{ref}(y_t^c|\tau)P_{ref}(\tau)} - \beta\sum_t\log\frac{\sum_\tau P(y_t^r|\tau)P(\tau)}{\sum_\tau P_{ref}(y_t^r|\tau)P_{ref}(\tau)}\right]$$
        - Weighted-sum the temperature and the generated token, marginalize the temperature.
- Experiment Results

  By training this module on a frozen Llama 3.0-8B-Instruct model, experimental results demonstrate the method's effectiveness:

  - Adaptive selection outperforms fixed parameters: In a mixed evaluation (UltraMathStories) encompassing math problems, creative writing, and general instructions, the sequence-level adaptive decoder achieved a higher win rate than all manually set fixed temperatures. It learned a distribution strategy with low temperatures for math problems and high temperatures for stories.
  - Achieving fine-grained token-level temperature control: In “constrained creative writing” tasks (e.g., writing a creative story where every sentence must begin with “Ab”). The token-level adaptive decoder demonstrated remarkable flexibility: it automatically lowered temperature (average $\tau = 0.21$) on the first word of each sentence to strictly adhere to rules, while raising temperature (average $\tau = 0.55$) on the rest of the sentence to unleash creativity.
  - Elevating Complex Reasoning Capabilities: In the GSM8K “Majority Voting” test, the model leverages token-level dynamic temperature control to precisely identify which parts of the reasoning chain should be convergent and which should diverge. This generates higher-quality, diverse reasoning paths, further enhancing the final voting accuracy.
- Work Cited
  - Paper
  - NotebookLM
  - Chat History